Streamlit Course

From Basics to Model Deployment



A 2-Hour Comprehensive Training

Course Overview

Module 1: Introduction (30 min)

What is Streamlit?
Why use Streamlit?
Installation & Setup

Module 2: Core Basics (40 min)

Basic components
Layouts & containers
Hands-on practice

Module 3: ML Integration (30 min)

Loading trained models
User input handling
Displaying predictions

Module 4: Deployment (20 min)

Deployment options
Streamlit Cloud setup
Best practices

What is Streamlit?



Streamlit is an open-source Python framework that turns data scripts into shareable web applications in minutes. No frontend experience required!



Pure Python

Write apps using only Python — no HTML, CSS, or JavaScript needed



Instant Updates

See changes in real-time as you edit your script



Rich Components

Built-in widgets for charts, maps, tables, and more

Why Use Streamlit?

Fast Prototyping

Build MVPs in hours, not weeks

Easy Sharing

Share apps with a simple URL

No DevOps Needed

Deploy without server management

Great for ML/AI

Perfect for data science demos

Installation & Setup



Step 1: Install Streamlit

```
pip install streamlit
```



Step 2: Verify Installation

```
streamlit hello
```

This opens a demo app in your browser at localhost:8501

Your First Streamlit App

Create a file: app.py

```
import streamlit as st
st.title('My First Streamlit App')
st.write('Hello, Streamlit!')
name = st.text_input('Enter your name:')
if name:
    st.success(f'Welcome, {name}!')
```

Run the app:

```
streamlit run app.py
```

Core Streamlit Components

`st.write()`
Display text, data, charts

`st.title()`
Main title for your app

`st.header()`
Section headers

`st.text_input()`
Single-line text input

`st.selectbox()`
Dropdown selection

`st.slider()`
Numeric slider input

`st.button()`
Clickable button

`st.checkbox()`
Toggle checkbox

`st.dataframe()`
Interactive data table

`st.chart()`
Simple line/bar charts

Layouts & Containers

Organize your app with powerful layout options:

Columns

```
col1, col2 = st.columns(2)

with col1:
    st.write('Left')

with col2:
    st.write('Right')
```

Sidebar & Tabs

```
# Sidebar
st.sidebar.title('Menu')

# Tabs
tab1, tab2 = st.tabs(['A',
'B'])
with tab1:
    st.write('Tab A')
```

Hands-On Practice



Create a simple calculator app with:

Number inputs for two values
Operation selector (Add, Subtract, Multiply, Divide)
Display the result when button is clicked

Loading Trained ML Models



Common formats for ML models:

Pickle (.pkl)

```
model = pickle.load(open('model.pkl', 'rb'))
```

Joblib (.joblib)

```
model = joblib.load('model.joblib')
```

Simple ML App Example

Iris Flower Classifier

```
import streamlit as st
import pickle

# Load model
model = pickle.load(open('iris_model.pkl', 'rb'))

st.title('Iris Flower Classifier')

# Input features
sepal_length = st.slider('Sepal Length', 4.0, 8.0, 5.0)
sepal_width = st.slider('Sepal Width', 2.0, 4.5, 3.0)

# Predict
if st.button('Predict'):
    prediction = model.predict([[sepal_length, sepal_width, ...]])
    st.success(f'Predicted: {prediction[0]}')
```

Caching for Performance



Use @st.cache_resource for models and data

Without caching, the model loads on every user interaction. Caching loads it once and reuses it.

```
@st.cache_resource
def load_model():
    return pickle.load(open('model.pkl', 'rb'))

# Load once, reuse across sessions
model = load_model()
```

Project File Structure



Recommended structure for deployment:

```
my-streamlit-app/
├── app.py                # Main application
├── requirements.txt      # Python dependencies
├── model.pkl             # Trained model file
├── data/                 # Data files (optional)
├── .streamlit/           # Config folder
│   └── config.toml       # App configuration
└── README.md             # Project documentation
```

Requirements.txt

List all Python packages your app needs:

```
streamlit==1.30.0
pandas==2.0.0
numpy==1.24.0
scikit-learn==1.3.0
matplotlib==3.7.0
plotly==5.14.0
```

Generate automatically:

```
pip install pip-chill
pip-chill > requirements.txt
```

Deployment Options



Streamlit Cloud

✓ Free, Easy, GitHub integration

✗ Limited resources



Heroku

✓ Flexible, Custom domains

✗ Paid plans recommended



AWS/GCP/Azure

✓ Scalable, Enterprise-ready

✗ More complex setup

Deploy to Streamlit Cloud

1. **Push to GitHub**
   Upload your app files to a GitHub repository

2. **Sign in to Streamlit Cloud**
   Go to share.streamlit.io and sign in with GitHub

3. **Deploy app**
   Click 'New app', select your repo, branch, and main file

4. **Configure (optional)**
   Set environment variables, secrets if needed

5. **Launch!**
   Your app gets a unique URL to share



Best Practices

- Use @st.cache_resource for models and expensive operations
- Keep your app.py clean and organized
- Add error handling for user inputs
- Include helpful descriptions for widgets
- Test locally before deploying
- Use .gitignore to exclude large files
- Add a README.md with setup instructions
- Monitor app performance and logs

Common Issues & Solutions

Issue: App keeps rerunning

Solution: Use st.cache or st.session_state to persist data

Issue: Model file too large for GitHub

Solution: Use Git LFS or store model externally (S3, Google Drive)

Issue: Slow app performance

Solution: Implement caching, reduce data processing, optimize model

Issue: Dependencies not installing

Solution: Check requirements.txt versions and compatibility

Useful Resources

Official Documentation — *docs.streamlit.io*
Complete API reference and guides

Streamlit Gallery — *streamlit.io/gallery*
Example apps for inspiration

Community Forum — *discuss.streamlit.io*
Get help from the community

Streamlit Components — *streamlit.io/components*
Extend functionality with custom components

30 Days of Streamlit — *30days.streamlit.app*
Daily challenges to learn Streamlit

Thank You!



Start Building Amazing Apps!

Questions?